CO4-AT1-CSA0925 - JAVA PROGRAMMING

I. MD IMTHIAZ
192521360

QUESTION 1: EMPLOYEE PAYROLLDASHBOARD

Code:

```
import java.awt.*;
import java.awt.event.*;
import java.util.ArrayList;
public class EmployeePayrollDashboard extends Frame
implements ActionListener {

    private TextField tfEmpId, tfEmpName, tfBasicSalary,
tfWorkingDays;
    private Choice choiceDept, choiceDesignation;
    private CheckboxGroup ratingGroup
```

```
private Checkbox cbExcellent, cbGood, cbAverage,
cbPoor;

    private Checkbox cbHRA, cbDA, cbMedical;


    // ---- Output / action components ----
    private List employeeList;
    private TextArea reportArea;
    private Button btnGenerate, btnReset, btnExit;

    private ArrayList<String> employeeRecords = new
 ArrayList<>();


    public EmployeePayrollDashboard() {
        super("Employee Performance & Payroll Dashboard");
        setLayout(new BorderLayout(10, 10));


        add(buildFormPanel(), BorderLayout.NORTH);
        add(buildListPanel(), BorderLayout.WEST);
```

```
    add(buildReportPanel(), BorderLayout.CENTER);
    add(buildButtonPanel(), BorderLayout.SOUTH);

    // Allow closing the window via the OS close button
    addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent e) {
            dispose();
            System.exit(0);
        }
    });

    setSize(760, 620);
    setLocationRelativeTo(null);
    setVisible(true);
}

private Panel buildFormPanel() {
    Panel formPanel = new Panel(new GridBagLayout());
    formPanel.setBackground(new Color(235, 242, 250));
```

```
GridBagConstraints gbc = new GridBagConstraints();
gbc.insets = new Insets(6, 8, 6, 8);
gbc.anchor = GridBagConstraints.WEST;
gbc.fill = GridBagConstraints.HORIZONTAL;

int row = 0;

gbc.gridx = 0; gbc.gridy = row;
formPanel.add(new Label("Employee ID:"), gbc);
tfEmpId = new TextField(15);
gbc.gridx = 1; formPanel.add(tfEmpId, gbc);

gbc.gridx = 2;
formPanel.add(new Label("Employee Name:"), gbc);
tfEmpName = new TextField(15);
gbc.gridx = 3; formPanel.add(tfEmpName, gbc);
row++;

// Department (Choice)
gbc.gridx = 0; gbc.gridy = row;
formPanel.add(new Label("Department:"), gbc);
```

```
choiceDept = new Choice();
choiceDept.add("IT");
choiceDept.add("HR");
choiceDept.add("Finance");
choiceDept.add("Operations");
choiceDept.add("Sales");
gbc.gridx = 1; formPanel.add(choiceDept, gbc);

// Designation (Choice)
gbc.gridx = 2;
formPanel.add(new Label("Designation:"), gbc);
choiceDesignation = new Choice();
choiceDesignation.add("Software Engineer");
choiceDesignation.add("Senior Engineer");
choiceDesignation.add("Team Lead");
choiceDesignation.add("Manager");
choiceDesignation.add("Intern");
gbc.gridx = 3; formPanel.add(choiceDesignation, gbc);
row++;

// Basic Salary
gbc.gridx = 0; gbc.gridy = row;
formPanel.add(new Label("Basic Salary (Rs.):"), gbc);
```

```
        tfBasicSalary = new TextField(15);
        gbc.gridx = 1; formPanel.add(tfBasicSalary, gbc);

        // Working Days
        gbc.gridx = 2;
        formPanel.add(new Label("Working Days:"), gbc);
        tfWorkingDays = new TextField(15);
        gbc.gridx = 3; formPanel.add(tfWorkingDays, gbc);
        row++;

        // Performance Rating (CheckboxGroup = radio buttons)
        gbc.gridx = 0; gbc.gridy = row;
        formPanel.add(new Label("Performance Rating:"), gbc);

        Panel ratingPanel = new Panel(new
FlowLayout(FlowLayout.LEFT, 8, 0));
        ratingGroup = new CheckboxGroup();
        cbExcellent = new Checkbox("Excellent", ratingGroup,
true);
        cbGood = new Checkbox("Good", ratingGroup, false);
        cbAverage = new Checkbox("Average", ratingGroup,
false);
        cbPoor = new Checkbox("Poor", ratingGroup, false);
        ratingPanel.add(cbExcellent);
```

```
        ratingPanel.add(cbGood);
        ratingPanel.add(cbAverage);
        ratingPanel.add(cbPoor);

        gbc.gridx = 1; gbc.gridwidth = 3;
        formPanel.add(ratingPanel, gbc);
        gbc.gridwidth = 1;
        row++;

        // Allowances (independent Checkboxes)
        gbc.gridx = 0; gbc.gridy = row;
        formPanel.add(new Label("Allowances:"), gbc);

        Panel allowancePanel = new Panel(new
FlowLayout(FlowLayout.LEFT, 8, 0));
        cbHRA = new Checkbox("HRA (20%)", true);
        cbDA = new Checkbox("DA (10%)", true);
        cbMedical = new Checkbox("Medical (Rs.1000)", true);
        allowancePanel.add(cbHRA);
        allowancePanel.add(cbDA);
        allowancePanel.add(cbMedical);

        gbc.gridx = 1; gbc.gridwidth = 3;
```

```
        formPanel.add(allowancePanel, gbc);
        gbc.gridwidth = 1;

        return formPanel;
    }

    private Panel buildListPanel() {
        Panel panel = new Panel(new BorderLayout(5, 5));
        panel.add(new Label("Employee Records:"),
BorderLayout.NORTH);
        employeeList = new List(10, false);
        panel.add(employeeList, BorderLayout.CENTER);
        panel.setPreferredSize(new Dimension(220, 200));
        return panel;
    }

    private Panel buildReportPanel() {
        Panel panel = new Panel(new BorderLayout(5, 5));
        panel.add(new Label("Payroll Report:"),
BorderLayout.NORTH);
        reportArea = new TextArea(15, 50);
        reportArea.setEditable(false);
        reportArea.setFont(new Font("Monospaced",
Font.PLAIN, 12));
```

```
        panel.add(reportArea, BorderLayout.CENTER);
        return panel;
    }

    private Panel buildButtonPanel() {
        Panel panel = new Panel(new
FlowLayout(FlowLayout.CENTER, 15, 10));
        btnGenerate = new Button("Generate Report");
        btnReset = new Button("Reset");
        btnExit = new Button("Exit");

        btnGenerate.addActionListener(this);
        btnReset.addActionListener(this);
        btnExit.addActionListener(this);

        panel.add(btnGenerate);
        panel.add(btnReset);
        panel.add(btnExit);
        return panel;
    }

    // ----------------------------------------
    // Event handling
```

```
// ----------------------------------------

@Override
public void actionPerformed(ActionEvent e) {
    Object src = e.getSource();
    if (src == btnGenerate) {
        generateReport();
    } else if (src == btnReset) {
        resetForm();
    } else if (src == btnExit) {
        dispose();
        System.exit(0);
    }
}

private void generateReport() {
    String empId = tfEmpId.getText().trim();
    String empName = tfEmpName.getText().trim();
    String basicText = tfBasicSalary.getText().trim();
    String daysText = tfWorkingDays.getText().trim();

    // ---- Validation ----
```

```
        if (empId.isEmpty() || empName.isEmpty() ||
basicText.isEmpty() || daysText.isEmpty()) {
            showMessage("Validation Error: Please fill in all fields
(ID, Name, Salary, Working Days).");
            return;
        }

        double basicSalary;
        int workingDays;
        try {
            basicSalary = Double.parseDouble(basicText);
            if (basicSalary <= 0) {
                showMessage("Validation Error: Basic Salary must
be a positive number.");
                return;
            }
        } catch (NumberFormatException ex) {
            showMessage("Validation Error: Basic Salary must be
numeric.");
            return;
        }

        try {
            workingDays = Integer.parseInt(daysText);
```

```
            if (workingDays < 0 || workingDays > 31) {
                showMessage("Validation Error: Working Days must be between 0 and 31.");
                return;
            }
        } catch (NumberFormatException ex) {
            showMessage("Validation Error: Working Days must be a whole number.");
            return;
        }

        String department = choiceDept.getSelectedItem();
        String designation = choiceDesignation.getSelectedItem();
        String rating = ratingGroup.getSelectedCheckbox().getLabel();

        // ---- Business rule calculations ----
        double proRatedBasic = (workingDays < 26)
                ? (basicSalary / 26.0) * workingDays
                : basicSalary;

        double hra = cbHRA.getState() ? proRatedBasic * 0.20 : 0;
```

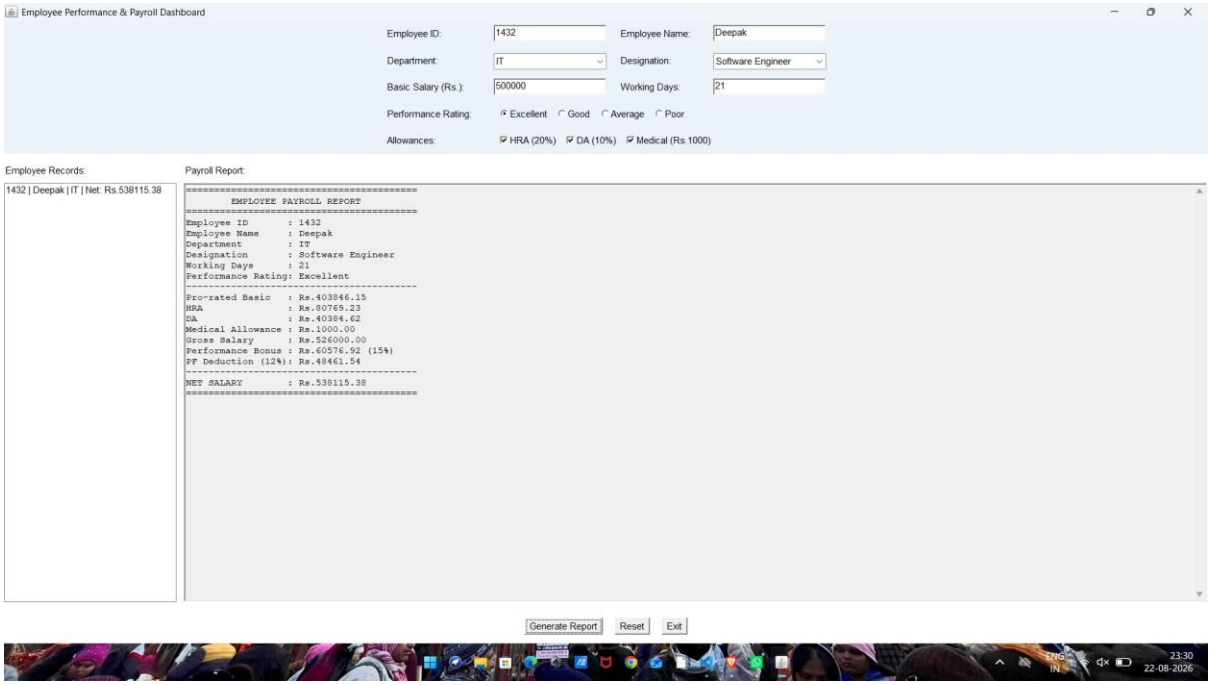
```
        double da = cbDA.getState() ? proRatedBasic * 0.10 : 0;
        double medical = cbMedical.getState() ? 1000 : 0;
        double grossSalary = proRatedBasic + hra + da +
medical;


        double bonusPercent;
        switch (rating) {
            case "Excellent": bonusPercent = 0.15; break;
            case "Good":      bonusPercent = 0.10; break;
            case "Average":   bonusPercent = 0.05; break;
            default:          bonusPercent = 0.0;
        }
        double performanceBonus = proRatedBasic *
bonusPercent;


        double pfDeduction = proRatedBasic * 0.12;
        double netSalary = grossSalary + performanceBonus -
pfDeduction;


        // ---- Build report entry ----
        String summary = String.format("%s | %s | %s | Net:
Rs.%.2f", empId, empName, department, netSalary);
        employeeRecords.add(summary);
        employeeList.add(summary);
```

```
        StringBuilder sb = new StringBuilder();
        sb.append("================================
=========\n");
        sb.append("        EMPLOYEE PAYROLL REPORT\n");
        sb.append("================================
=========\n");
        sb.append(String.format("Employee ID       : %s%n",
empId));
        sb.append(String.format("Employee Name     : %s%n",
empName));
        sb.append(String.format("Department        : %s%n",
department));
        sb.append(String.format("Designation       : %s%n",
designation));
        sb.append(String.format("Working Days      : %d%n",
workingDays));
        sb.append(String.format("Performance Rating: %s%n",
rating));
        sb.append("--------------------------\n");
        sb.append(String.format("Pro-rated Basic   :
Rs.%.2f%n", proRatedBasic));
        sb.append(String.format("HRA               : Rs.%.2f%n",
hra));
        sb.append(String.format("DA                : Rs.%.2f%n",
da));
```

```
        sb.append(String.format("Medical Allowance :
Rs.%.2f%n", medical));
        sb.append(String.format("Gross Salary      : Rs.%.2f%n",
grossSalary));
        sb.append(String.format("Performance Bonus : Rs.%.2f
(%.0f%%)%n", performanceBonus, bonusPercent * 100));
        sb.append(String.format("PF Deduction (12%%):
Rs.%.2f%n", pfDeduction));
        sb.append("----------------------------\n");
        sb.append(String.format("NET SALARY       :
Rs.%.2f%n", netSalary));
        sb.append("================================
=========\n\n");

        reportArea.append(sb.toString());
        reportArea.setCaretPosition(reportArea.getText().length(
));
    }

    private void resetForm() {
        tfEmpId.setText("");
        tfEmpName.setText("");
        tfBasicSalary.setText("");
        tfWorkingDays.setText("");
        choiceDept.select(0);
```

```
        choiceDesignation.select(0);
        ratingGroup.setSelectedCheckbox(cbExcellent);
        cbHRA.setState(true);
        cbDA.setState(true);
        cbMedical.setState(true);
        tfEmpId.requestFocus();
        // Note: Reset only clears the input form.
        // employeeList / reportArea history is intentionally kept.
    }

    private void showMessage(String msg) {
        reportArea.append(">> " + msg + "\n\n");
        reportArea.setCaretPosition(reportArea.getText().length(
));
    }

    public static void main(String[] args) {
        EventQueue.invokeLater(EmployeePayrollDashboard::n
ew);
    }
}
```

OUTPUT:



QUESTION 2 – ONLINE FOOD ORDER AND BILL GENRATING SYSTEM

CODE:

```
import java.awt.*;
import java.awt.event.*;
import java.util.HashMap;
import java.util.Map;

public class FoodOrderingBillingSystem extends Frame implements ActionListener {

    // ---- CardLayout management ----
    private CardLayout cardLayout;
    private Panel cardPanel;
```

```
private static final String CARD_CUSTOMER = "Customer/Order Details";
private static final String CARD_FOOD = "Food Selection";
private static final String CARD_BILL = "Bill/Order Summary";

// ---- Card 1: Customer/Order Details ----
private TextField tfCustomerName, tfTableNumber;

// ---- Card 2: Food Selection ----
private Choice choiceCategory, choiceFoodItem;
private TextField tfQuantity;
private List orderList;
private Button btnAddItem;

// ---- Card 3: Bill/Order Summary ----
private TextArea billArea;
private Button btnGenerateBill, btnClear;

// ---- Navigation buttons (shared) ----
private Button btnNext1, btnNext2, btnPrevious2, btnPrevious3;

// ---- Menu data: category -> {item -> price} ----
private Map<String, Map<String, Double>> menu = new HashMap<>();

// ---- Order data ----
private double subtotal = 0.0;
private static final double GST_RATE = 0.05;

public FoodOrderingBillingSystem() {
    super("Online Food Ordering and Billing System");
    buildMenuData();
```

```
    cardLayout = new CardLayout();
    cardPanel = new Panel(cardLayout);

    cardPanel.add(buildCustomerCard(), CARD_CUSTOMER);
    cardPanel.add(buildFoodSelectionCard(), CARD_FOOD);
    cardPanel.add(buildBillCard(), CARD_BILL);

    setLayout(new BorderLayout());
    add(cardPanel, BorderLayout.CENTER);

    addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent e) {
            dispose();
            System.exit(0);
        }
    });

    setSize(700, 520);
    setLocationRelativeTo(null);
    setVisible(true);
}

// ----------------------------------------
// Menu data setup
// ----------------------------------------

private void buildMenuData() {
    Map<String, Double> starters = new HashMap<>();
    starters.put("Spring Roll", 120.0);
```

```
    starters.put("Paneer Tikka", 180.0);
    starters.put("Chicken Wings", 220.0);
    menu.put("Starters", starters);

    Map<String, Double> mainCourse = new HashMap<>();
    mainCourse.put("Veg Biryani", 180.0);
    mainCourse.put("Chicken Biryani", 250.0);
    mainCourse.put("Butter Naan", 40.0);
    mainCourse.put("Paneer Butter Masala", 220.0);
    menu.put("Main Course", mainCourse);

    Map<String, Double> beverages = new HashMap<>();
    beverages.put("Cold Coffee", 90.0);
    beverages.put("Fresh Lime Soda", 60.0);
    beverages.put("Mango Lassi", 80.0);
    menu.put("Beverages", beverages);

    Map<String, Double> desserts = new HashMap<>();
    desserts.put("Gulab Jamun", 70.0);
    desserts.put("Ice Cream", 90.0);
    desserts.put("Brownie", 110.0);
    menu.put("Desserts", desserts);
}

// ----------------------------------------
// Card 1: Customer / Order Details
// ----------------------------------------

private Panel buildCustomerCard() {
    Panel outer = new Panel(new BorderLayout());
```

```
Panel form = new Panel(new GridBagLayout());
GridBagConstraints gbc = new GridBagConstraints();
gbc.insets = new Insets(10, 10, 10, 10);
gbc.anchor = GridBagConstraints.WEST;
gbc.fill = GridBagConstraints.HORIZONTAL;

Label heading = new Label("STEP 1: Customer / Order Details", Label.CENTER);
heading.setFont(new Font("SansSerif", Font.BOLD, 16));
gbc.gridx = 0; gbc.gridy = 0; gbc.gridwidth = 2;
form.add(heading, gbc);
gbc.gridwidth = 1;

gbc.gridx = 0; gbc.gridy = 1;
form.add(new Label("Customer Name:"), gbc);
tfCustomerName = new TextField(20);
gbc.gridx = 1;
form.add(tfCustomerName, gbc);

gbc.gridx = 0; gbc.gridy = 2;
form.add(new Label("Table Number:"), gbc);
tfTableNumber = new TextField(20);
gbc.gridx = 1;
form.add(tfTableNumber, gbc);

outer.add(form, BorderLayout.CENTER);

Panel navPanel = new Panel(new FlowLayout(FlowLayout.RIGHT));
btnNext1 = new Button("Next >>");
btnNext1.addActionListener(this);
```

```
    navPanel.add(btnNext1);
    outer.add(navPanel, BorderLayout.SOUTH);

    return outer;
}

// ----------------------------------------
// Card 2: Food Selection
// ----------------------------------------

private Panel buildFoodSelectionCard() {
    Panel outer = new Panel(new BorderLayout(5, 5));

    Panel form = new Panel(new GridBagLayout());
    GridBagConstraints gbc = new GridBagConstraints();
    gbc.insets = new Insets(8, 8, 8, 8);
    gbc.anchor = GridBagConstraints.WEST;
    gbc.fill = GridBagConstraints.HORIZONTAL;

    Label heading = new Label("STEP 2: Food Selection", Label.CENTER);
    heading.setFont(new Font("SansSerif", Font.BOLD, 16));
    gbc.gridx = 0; gbc.gridy = 0; gbc.gridwidth = 2;
    form.add(heading, gbc);
    gbc.gridwidth = 1;

    gbc.gridx = 0; gbc.gridy = 1;
    form.add(new Label("Food Category:"), gbc);
    choiceCategory = new Choice();
    for (String cat : menu.keySet()) {
        choiceCategory.add(cat);
```

```
}
choiceCategory.addItemListener(e -> populateFoodItems());
gbc.gridx = 1;
form.add(choiceCategory, gbc);

gbc.gridx = 0; gbc.gridy = 2;
form.add(new Label("Food Item:"), gbc);
choiceFoodItem = new Choice();
gbc.gridx = 1;
form.add(choiceFoodItem, gbc);

gbc.gridx = 0; gbc.gridy = 3;
form.add(new Label("Quantity:"), gbc);
tfQuantity = new TextField("1", 20);
gbc.gridx = 1;
form.add(tfQuantity, gbc);

btnAddItem = new Button("Add Item");
btnAddItem.addActionListener(this);
gbc.gridx = 1; gbc.gridy = 4;
form.add(btnAddItem, gbc);

outer.add(form, BorderLayout.NORTH);

Panel listPanel = new Panel(new BorderLayout(5, 5));
listPanel.add(new Label("Current Order:"), BorderLayout.NORTH);
orderList = new List(8, false);
listPanel.add(orderList, BorderLayout.CENTER);
outer.add(listPanel, BorderLayout.CENTER);
```

```
    Panel navPanel = new Panel(new FlowLayout(FlowLayout.RIGHT));
    btnPrevious2 = new Button("<< Previous");
    btnNext2 = new Button("Next >>");
    btnPrevious2.addActionListener(this);
    btnNext2.addActionListener(this);
    navPanel.add(btnPrevious2);
    navPanel.add(btnNext2);
    outer.add(navPanel, BorderLayout.SOUTH);

    // populate items for the first category by default
    populateFoodItems();

    return outer;
}

private void populateFoodItems() {
    choiceFoodItem.removeAll();
    String category = choiceCategory.getSelectedItem();
    Map<String, Double> items = menu.get(category);
    if (items != null) {
        for (String item : items.keySet()) {
            choiceFoodItem.add(item);
        }
    }
}

// ----------------------------------------
// Card 3: Bill / Order Summary
// ----------------------------------------
```

```
private Panel buildBillCard() {
    Panel outer = new Panel(new BorderLayout(5, 5));

    Label heading = new Label("STEP 3: Bill / Order Summary", Label.CENTER);
    heading.setFont(new Font("SansSerif", Font.BOLD, 16));
    outer.add(heading, BorderLayout.NORTH);

    billArea = new TextArea(18, 55);
    billArea.setEditable(false);
    billArea.setFont(new Font("Monospaced", Font.PLAIN, 12));
    outer.add(billArea, BorderLayout.CENTER);

    Panel buttonPanel = new Panel(new FlowLayout(FlowLayout.CENTER, 15, 10));
    btnPrevious3 = new Button("<< Previous");
    btnGenerateBill = new Button("Generate Bill");
    btnClear = new Button("Clear");

    btnPrevious3.addActionListener(this);
    btnGenerateBill.addActionListener(this);
    btnClear.addActionListener(this);

    buttonPanel.add(btnPrevious3);
    buttonPanel.add(btnGenerateBill);
    buttonPanel.add(btnClear);
    outer.add(buttonPanel, BorderLayout.SOUTH);

    return outer;
}

// ----------------------------------------
```

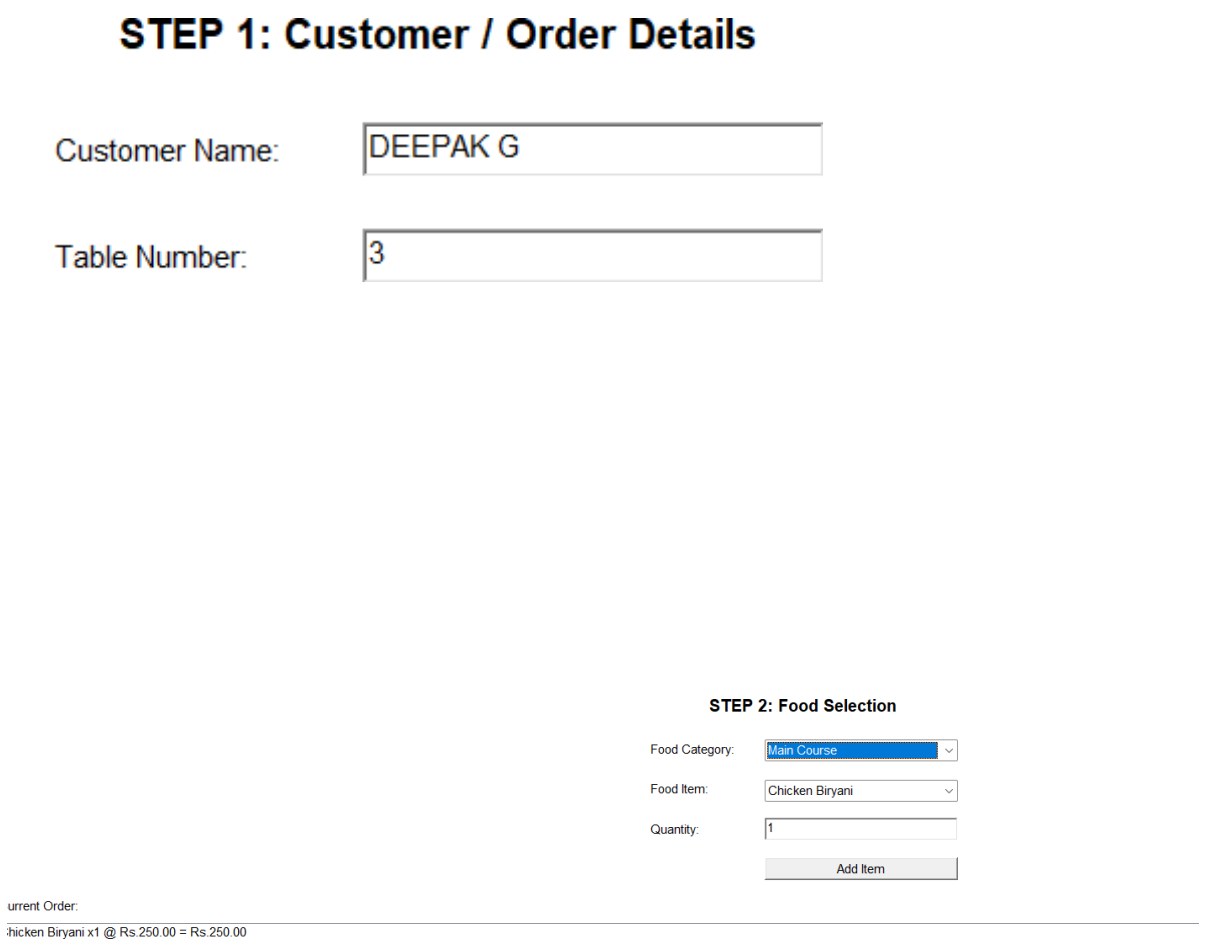
```
    // Event handling
    // ----------------------------------------

    @Override
    public void actionPerformed(ActionEvent e) {
        Object src = e.getSource();

        if (src == btnNext1) {
            if (tfCustomerName.getText().trim().isEmpty() ||
tfTableNumber.getText().trim().isEmpty()) {
                showDialogMessage("Please enter Customer Name and Table Number before
proceeding.");
                return;
            }
            cardLayout.show(cardPanel, CARD_FOOD);

        } else if (src == btnAddItem) {
            addItemToOrder();

        } else if (src == btnPrevious2) {
            cardLayout.show(cardPanel, CARD_CUSTOMER);

        } else if (src == btnNext2) {
            if (orderList.getItemCount() == 0) {
                showDialogMessage("Please add at least one item before generating the bill.");
                return;
            }
            cardLayout.show(cardPanel, CARD_BILL);

        } else if (src == btnPrevious3) {
            cardLayout.show(cardPanel, CARD_FOOD);
```

```
    } else if (src == btnGenerateBill) {
        generateBill();

    } else if (src == btnClear) {
        clearOrder();
    }
}

private void addItemToOrder() {
    String category = choiceCategory.getSelectedItem();
    String item = choiceFoodItem.getSelectedItem();

    if (item == null || item.isEmpty()) {
        showDialogMessage("Please select a food item before adding.");
        return;
    }

    String qtyText = tfQuantity.getText().trim();
    int quantity;
    try {
        quantity = Integer.parseInt(qtyText);
        if (quantity <= 0) {
            showDialogMessage("Quantity must be a positive whole number.");
            return;
        }
    } catch (NumberFormatException ex) {
        showDialogMessage("Quantity must be a valid whole number.");
        return;
    }
```

```
        double price = menu.get(category).get(item);
        double itemCost = price * quantity;
        subtotal += itemCost;

        String line = String.format("%s x%d @ Rs.%.2f = Rs.%.2f", item, quantity, price,
itemCost);
        orderList.add(line);

        tfQuantity.setText("1");
    }

    private void generateBill() {
        double gst = subtotal * GST_RATE;
        double finalAmount = subtotal + gst;

        StringBuilder sb = new StringBuilder();
        sb.append("=========================================\n");
        sb.append("           RESTAURANT FINAL BILL\n");
        sb.append("=========================================\n");
        sb.append(String.format("Customer Name : %s%n", tfCustomerName.getText().trim()));
        sb.append(String.format("Table Number  : %s%n", tfTableNumber.getText().trim()));
        sb.append("----------------------------\n");
        sb.append("Ordered Items:\n");
        String[] items = orderList.getItems();
        for (String item : items) {
            sb.append("  - ").append(item).append("\n");
        }
        sb.append("----------------------------\n");
        sb.append(String.format("Subtotal       : Rs.%.2f%n", subtotal));
        sb.append(String.format("GST (5%%)      : Rs.%.2f%n", gst));
```

```
    sb.append("----------------------------\n");
    sb.append(String.format("FINAL AMOUNT  : Rs.%.2f%n", finalAmount));
    sb.append("========================================\n");

    billArea.setText(sb.toString());
}

private void clearOrder() {
    orderList.removeAll();
    billArea.setText("");
    subtotal = 0.0;
    tfCustomerName.setText("");
    tfTableNumber.setText("");
    tfQuantity.setText("1");
    cardLayout.show(cardPanel, CARD_CUSTOMER);
}

private void showDialogMessage(String msg) {
    // Simple AWT message dialog using a Frame-based Dialog
    Dialog dialog = new Dialog(this, "Notice", true);
    dialog.setLayout(new BorderLayout(10, 10));
    dialog.add(new Label(msg, Label.CENTER), BorderLayout.CENTER);
    Button ok = new Button("OK");
    ok.addActionListener(ev -> dialog.dispose());
    Panel p = new Panel();
    p.add(ok);
    dialog.add(p, BorderLayout.SOUTH);
    dialog.setSize(350, 130);
    dialog.setLocationRelativeTo(this);
    dialog.setVisible(true);
```

```
    }

    public static void main(String[] args) {
        EventQueue.invokeLater(FoodOrderingBillingSystem::new);
    }
}
```



Online Food Ordering and Billing System

STEP 3: Bill / Order Summary

```
========================================
          RESTAURANT FINAL BILL
========================================
ıstomer Name : DEEPAK G
ıble Number  : 3
----------------------------------------
:dered Items:
 - Chicken Biryani x1 @ Rs.250.00 = Rs.250.00
----------------------------------------
ıbtotal      : Rs.250.00
ST (5%)      : Rs.12.50
----------------------------------------
INAL AMOUNT  : Rs.262.50
========================================
```